

Homework #2 Solution: STA5021

통계학과 신현수 2025712931

2026-04-11

Problem 1

(10 points) Verify that the degrees of freedom of a spline function with n knots (possibly with multiple knots at interior breakpoints) and order m is $n - 2 + m$.

[Solution]

서로 다른 breakpoint를 $b_0 < b_1 < \dots < b_L$ 이라 하자. 주어진 spline은 총 L 개의 구간 $(b_0, b_1), (b_1, b_2), \dots, (b_{L-1}, b_L)$ 에서 정의된다.

Interior breakpoint b_j ($j = 1, \dots, L-1$)에서의 knot multiplicity를 q_j 라 하면, 전체 knot의 수 n 은 다음과 같다.

$$n = 2 + \sum_{j=1}^{L-1} q_j$$

Order m 인 spline은 L 개의 각 구간에서 m 개의 계수를 가지므로, 제약 없는 전체 모수의 수는 mL 이다. Interior breakpoint b_j 에서 multiplicity q_j 인 knot은 $(m-1-q_j)$ 차 도함수까지의 연속성을 요구하므로 $m-q_j$ 개의 제약 조건을 부과한다. 따라서 자유도 df 는 다음과 같이 계산된다.

$$\begin{aligned} df &= mL - \sum_{j=1}^{L-1} (m - q_j) \\ &= mL - m(L-1) + \sum_{j=1}^{L-1} q_j \\ &= m + \sum_{j=1}^{L-1} q_j \\ &= m + (n-2) \\ &= n-2+m \end{aligned}$$

따라서, 자유도는 $n - 2 + m$ 임이 증명된다.

Problem 2

(20 points) In Slide 3-1, we saw a method for smoothing a positive function. To do this, we seek $\mathbf{c}$ that minimizes

$$F(\mathbf{c}) = (\mathbf{y} - e^{\Phi\mathbf{c}})^\top (\mathbf{y} - e^{\Phi\mathbf{c}}) + \lambda \int [D^2(\mathbf{c}^\top \phi(t))]^2 dt.$$

To apply the Newton-Raphson algorithm, we need the gradient $\partial F(\mathbf{c})/\partial \mathbf{c}$. Show that

$$\frac{\partial F(\mathbf{c})}{\partial \mathbf{c}} = -2\Phi^\top [(\mathbf{y} - e^{\Phi\mathbf{c}}) \odot e^{\Phi\mathbf{c}}] + 2\lambda \mathbf{R}\mathbf{c},$$

where $\odot$ is a component-wise multiplication.

[Solution]

$W(t) = \mathbf{c}^\top \phi(t)$, $x(t) = \exp\{W(t)\}$ 라 하자.

목적함수 $F(\mathbf{c})$ 는 데이터 적합항 $F_1(\mathbf{c})$ 와 페널티항 $F_2(\mathbf{c})$ 로 나눌 수 있다.

$$F(\mathbf{c}) = \underbrace{(\mathbf{y} - e^{\Phi\mathbf{c}})^\top (\mathbf{y} - e^{\Phi\mathbf{c}})}_{F_1(\mathbf{c})} + \underbrace{\lambda \int [D^2(\mathbf{c}^\top \phi(t))]^2 dt}_{F_2(\mathbf{c})}$$

여기서 roughness penalty matrix $\mathbf{R} = (\int D^2 \phi_k(t) D^2 \phi_\ell(t) dt)_{k\ell}$ 은 대칭행렬이며, 페널티항은 $\lambda \mathbf{c}^\top \mathbf{R} \mathbf{c}$ 가 된다.

(1) $F_1(\mathbf{c})$ 의 미분

$$F_1(\mathbf{c}) = \sum_{i=1}^n (y_i - e^{(\Phi\mathbf{c})_i})^2 \text{ 이다.}$$

$$z_i = (\Phi\mathbf{c})_i = \phi(t_i)^\top \mathbf{c} \text{ 로 두면 } \frac{\partial z_i}{\partial \mathbf{c}} = \phi(t_i) \text{ 이다.}$$

Chain rule에 의해,

$$\begin{aligned} \frac{\partial F_1(\mathbf{c})}{\partial \mathbf{c}} &= \sum_{i=1}^n 2(y_i - e^{z_i}) \frac{\partial}{\partial \mathbf{c}} (y_i - e^{z_i}) \\ &= \sum_{i=1}^n 2(y_i - e^{z_i}) (-e^{z_i}) \phi(t_i) \\ &= -2 \sum_{i=1}^n (y_i - e^{z_i}) e^{z_i} \phi(t_i) \end{aligned}$$

이를 행렬 형태로 정리하면 다음과 같다.

$$\frac{\partial F_1(\mathbf{c})}{\partial \mathbf{c}} = -2\Phi^\top \left((\mathbf{y} - e^{\Phi\mathbf{c}}) \odot e^{\Phi\mathbf{c}} \right)$$

(2) $F_2(\mathbf{c})$ 의 미분

$F_2(\mathbf{c}) = \lambda \mathbf{c}^\top \mathbf{R} \mathbf{c}$ 이며, $\mathbf{R}$ 이 대칭행렬이므로 이차형식 미분 공식에 의해 다음과 같다.

$$\frac{\partial F_2(\mathbf{c})}{\partial \mathbf{c}} = 2\lambda \mathbf{R}\mathbf{c}$$

전체 그래디언트는 각 항의 미분의 합이므로,

$$\frac{\partial F(\mathbf{c})}{\partial \mathbf{c}} = -2\Phi^\top \left((\mathbf{y} - e^{\Phi \mathbf{c}}) \odot e^{\Phi \mathbf{c}} \right) + 2\lambda \mathbf{R} \mathbf{c}$$

가 성립한다.

Problem 3

(10 points) Use the fda package in R to complete the following tasks. Load the Berkeley Earth temperature data:

```
library(fda)

temperature <- read.table(
  "https://berkeley-earth-temperature.s3.us-west-1.amazonaws.com/Global/Land_and_Ocean_summary.txt",
  skip = 57
)
temperature <- temperature[, 1:2]
names(temperature) <- c("year", "temp")
```

- (a) Generate 100 cubic B-spline basis functions on the interval [1850, 2024] using equally spaced knots. How many knots are there?

[Solution]

100개의 cubic B-spline basis를 구성한다. fda 패키지에서 B-spline basis의 개수는 다음 관계를 만족한다.

$$\text{nbasis} = \text{norder} + \text{nbreaks} - 2$$

nbasis = 100, norder = 4 (cubic spline)이므로 nbreaks = 98이다. 즉, boundary를 포함한 knot의 수는 98개이며, interior knot의 수는 96개이다.

```
norder <- 4
nbasis <- 100
nbreaks <- nbasis - norder + 2 # 98

breaks <- seq(1850, 2024, length.out = nbreaks)

bb100 <- create.bspline.basis(
  rangeval = c(1850, 2024),
  nbasis = nbasis,
  norder = norder,
  breaks = breaks
)

cat("Number of breaks (boundary):", length(breaks), "\n")
```

```
## Number of breaks (boundary): 98
```

```
cat("Number of interior knots:", length(breaks) - 2, "\n")
```

```
## Number of interior knots: 96
```

- (b) Using the basis functions in (a), fit a smooth function to the Berkeley Earth temperature data by (i) least squares and (ii) a roughness penalty approach with $\lambda = 10$. Plot both fitted functions along with data points. Compare the (effective) degrees of freedom of the two approaches.

[Solution]

Least squares fit은 `smooth.basis()`의 세 번째 인자에 `basisfd` object를 직접 전달하면 되고, roughness penalty fit은 `fdPar()`로 `penalty`와 λ 를 지정한다.

- Least squares fit의 effective degrees of freedom은 basis 개수 K 와 같다 ($n \geq K$ 인 경우).
- Roughness penalty fit의 effective degrees of freedom은 hat matrix의 trace, 즉 $\text{tr}(S_{\phi, \lambda})$ 로 정의된다.

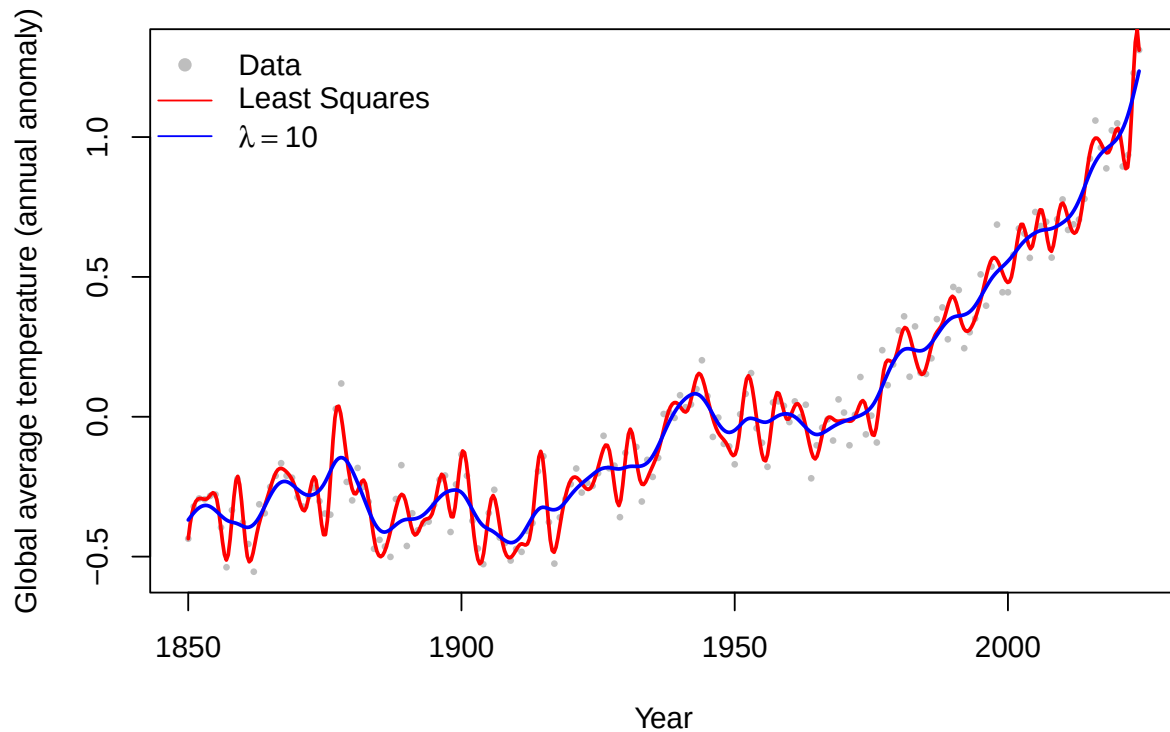
```
# (i) Least squares fit
fit.ls <- smooth.basis(
  argvals = temperature$year,
  y        = temperature$temp,
  fdParobj = bb100
)

# (ii) Roughness penalty fit (lambda = 10)
fit.pen10 <- smooth.basis(
  argvals = temperature$year,
  y        = temperature$temp,
  fdParobj = fdPar(bb100, Lfdobj = 2, lambda = 10)
)

#
yearfine <- seq(1850, 2024, length.out = 500)
yhat.ls  <- eval.fd(yearfine, fit.ls$fd)
yhat.p10 <- eval.fd(yearfine, fit.pen10$fd)

plot(temperature$year, temperature$temp,
     pch = 16, cex = 0.5, col = "gray",
     xlab = "Year", ylab = "Global average temperature (annual anomaly)",
     main = "Berkeley Earth Temperature Data Fit")
lines(yearfine, yhat.ls, lwd = 2, col = "red")
lines(yearfine, yhat.p10, lwd = 2, col = "blue")
legend("topleft",
     legend = c("Data", "Least Squares", expression(lambda == 10)),
     col = c("gray", "red", "blue"), lty = c(NA, 1, 1), pch = c(16, NA, NA),
     bty = "n")
```

Berkeley Earth Temperature Data Fit



```
# Effective degrees of freedom
cat("Degrees of Freedom (Least Squares):", fit.ls$df, "\n")
```

```
## Degrees of Freedom (Least Squares): 100
```

```
cat("Effective Degrees of Freedom (lambda=10):", fit.pen10$df, "\n")
```

```
## Effective Degrees of Freedom (lambda=10): 35.27691
```

Least squares fit의 effective df는 basis 개수인 100이다. Roughness penalty ($\lambda = 10$) fit의 effective df는 35.28로, λ 가 클수록 effective df는 감소한다. 이는 penalty가 함수의 복잡도를 억제하여 실질적으로 사용되는 자유도를 줄이기 때문이다.

- (c) Suppose we want to fit a cubic spline that allows an abrupt change at $t = 1965$. That is, the spline must remain continuous at $t = 1965$, but its first derivative need not be continuous. Generate 100 cubic B-spline basis functions on $[1850, 2024]$ and fit a function using a roughness penalty approach with $\lambda = 1000$. Plot the fitted function along with the data points. Can you observe an abrupt change of the fitted function at $t = 1965$?

[Solution]

```
# knot multiplicity = 3 at t = 1965
# + , 1
breaks1965 <- c(
  seq(1850, 1965, length.out = 48),
  1965, 1965, # x2 + multiplicity = 3
  seq(1965, 2024, length.out = 49)[-1]
)

cat("Length of breaks:", length(breaks1965), "\n")
```

```
## Length of breaks: 98
```

```
cat("Multiplicity at t=1965:", sum(breaks1965 == 1965), "\n")
```

```
## Multiplicity at t=1965: 3
```

```
bb1965 <- create.bspline.basis(
  rangeval = c(1850, 2024),
  norder = 4,
  breaks = breaks1965
)
cat("nbasis:", bb1965$nbasis, "\n")
```

```
## nbasis: 100
```

```
# Roughness penalty fit (lambda = 1000)
fit.kink <- smooth.basis(
  argvals = temperature$year,
  y = temperature$temp,
  fdParobj = fdPar(bb1965, Lfdobj = 2, lambda = 1000)
)

# : ordinary cubic spline (lambda = 1000)
fit.nokink <- smooth.basis(
  argvals = temperature$year,
  y = temperature$temp,
  fdParobj = fdPar(bb100, Lfdobj = 2, lambda = 1000)
)

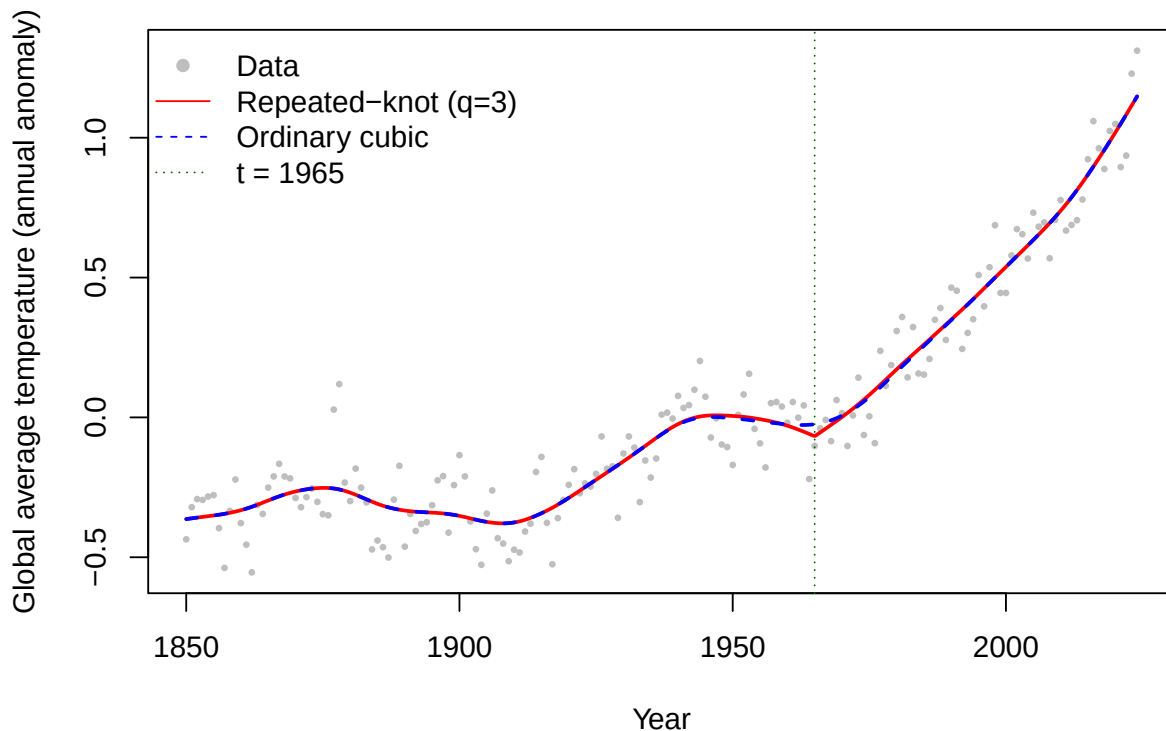
# 1: lambda=1000, repeated vs ordinary
yhat.kink <- eval.fd(yearfine, fit.kink$fd)
yhat.nokink <- eval.fd(yearfine, fit.nokink$fd)
```

```

plot(temperature$year, temperature$temp,
     pch = 16, cex = 0.5, col = "gray",
     xlab = "Year", ylab = "Global average temperature (annual anomaly)",
     main = expression("Cubic Spline with Repeated Knot (" * lambda == 1000 * ")"))
lines(yearfine, yhat.kink, lwd = 2, col = "red")
lines(yearfine, yhat.nokink, lwd = 2, col = "blue", lty = 2)
abline(v = 1965, lty = 3, col = "darkgreen")
legend("topleft",
       legend = c("Data", "Repeated-knot (q=3)", "Ordinary cubic", "t = 1965"),
       col = c("gray", "red", "blue", "darkgreen"),
       pch = c(16, NA, NA, NA),
       lty = c(NA, 1, 2, 3),
       bty = "n")

```

Cubic Spline with Repeated Knot ($\lambda = 1000$)



```

# 2: 1 (lambda=1000)
D1.kink <- eval.fd(yearfine, fit.kink$fd, Lfd = 1)
D1.nokink <- eval.fd(yearfine, fit.nokink$fd, Lfd = 1)

plot(yearfine, D1.kink,
     type = "l", lwd = 2, col = "red",
     xlab = "Year", ylab = "First derivative",
     main = expression("First Derivative (" * lambda == 1000 * ")"))
lines(yearfine, D1.nokink, lwd = 2, col = "blue", lty = 2)
abline(v = 1965, lty = 3, col = "darkgreen")

```

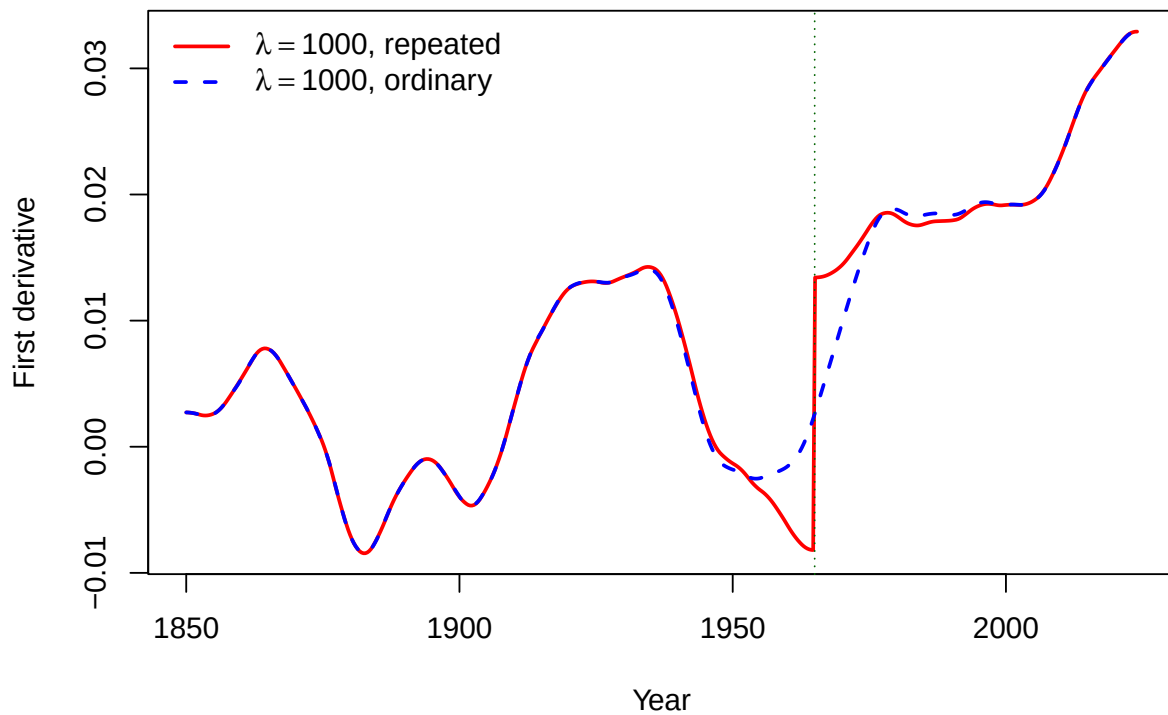


```

legend("topleft",
      legend = c(expression(lambda == 1000 * ", repeated"),
                  expression(lambda == 1000 * ", ordinary")),
      col = c("red", "blue"), lty = c(1, 2), lwd = 2, bty = "n")

```

First Derivative ($\lambda = 1000$)



$\lambda = 1000$ 의 강한 smoothing 하에서는 repeated-knot fit과 ordinary fit이 시각적으로 거의 일치하며, $t = 1965$ 에서 fitted curve 자체의 뚜렷한 abrupt change는 관찰되지 않는다. 다만 repeated knot를 사용했기 때문에 이론적으로는 $t = 1965$ 에서 slope change가 허용되며, 1차 도함수 플롯은 ordinary fit과 다른 국소적 변화 양상을 보여준다. 따라서 이 경우의 abrupt change는 함수값의 jump라기보다 기울기의 갑작스러운 변화(kink)로 해석하는 것이 적절하다.

Problem 4

(20 points) In the March 30 class, we implemented shift registration in R. Specifically, when $x^* = x \circ h$ and $h(t) = t + \delta$, the `shiftreg()` function finds δ that minimizes

$$F_{\text{shift}}(\delta) = \int_{T_1}^{T_2} (x^*(t) - x_0(t))^2 dt$$

Now suppose that $h(t) = a + bt$ for $a, b \in \mathbb{R}$, and define $x^* = x \circ h$. Using the criterion

$$F_{\text{linear}}(a, b) = \int_{T_1}^{T_2} (x^*(t) - x_0(t))^2 dt,$$

write an R function for a linear registration. The `linearreg()` function that returns a and b that minimize the above criterion should have the following interface:

```
linearreg(t0, x0, t, x, iter.max = 10)
```

[Solution]

이 문제는 shift registration의 $h(t) = t + \delta$ 를 $h(t) = a + bt$ 로 일반화한 것이다. $x^*(t) = x(h(t)) = x(a + bt)$ 로 두면 criterion은

$$F_{\text{linear}}(a, b) = \int_{T_1}^{T_2} \{x(a + bt) - x_0(t)\}^2 dt.$$

편의를 위해 $z(t) = a + bt$, $r(t) = x(z(t)) - x_0(t)$ 로 두면 $F_{\text{linear}}(a, b) = \int r(t)^2 dt$ 이다. 체인룰에 의해 $\partial z / \partial a = 1$, $\partial z / \partial b = t$ 이므로 **gradient**는

$$g(a, b) = \begin{pmatrix} 2 \int r(t) Dx(z(t)) dt \\ 2 \int t r(t) Dx(z(t)) dt \end{pmatrix}.$$

Exact Hessian은

$$H(a, b) = 2 \int \{[Dx(z(t))]^2 + r(t) D^2 x(z(t))\} \begin{pmatrix} 1 & t \\ t & t^2 \end{pmatrix} dt.$$

Shift registration 구현 슬라이드와 동일하게 $D^2 x$ 항을 무시하는 **practical Hessian** 근사를 사용한다.

$$\tilde{H}(a, b) = 2 \int [Dx(z(t))]^2 \begin{pmatrix} 1 & t \\ t & t^2 \end{pmatrix} dt.$$

Newton–Raphson update는 초기값 $(a^{(0)}, b^{(0)}) = (0, 1)$ (identity warping)에서 시작하여

$$\begin{pmatrix} a^{(\nu)} \\ b^{(\nu)} \end{pmatrix} = \begin{pmatrix} a^{(\nu-1)} \\ b^{(\nu-1)} \end{pmatrix} - \alpha \tilde{H}(a^{(\nu-1)}, b^{(\nu-1)})^{-1} g(a^{(\nu-1)}, b^{(\nu-1)})$$

로 반복한다. 여기서 $\alpha \in (0, 1]$ 은 step size이며, objective function이 감소하도록 step-halving으로 자동 조정된다. 또한 $h'(t) = b$ 이므로 시간 순서 보존을 위해 $b > 0$ 을 유지한다.

구현에서는 shift registration 슬라이드와 동일하게 세 가지 근사를 사용했다.

1. $x(a + bt)$: `approx()`를 이용한 **linear interpolation**으로 근사
2. $Dx(a + bt)$: 인접 점 기울기 $\frac{x(t_j) - x(t_{j-1})}{t_j - t_{j-1}}$ 로 근사
3. 적분: **trapezoidal rule**로 근사. $z = a + bt$ 가 관측 구간 밖이면 $x(z) = 0$ 으로 처리

```
# trapezoidal weights
trapw <- function(tt){
  n <- length(tt)
  c((tt[2] - tt[1]) / 2,
    (tt[3:n] - tt[1:(n - 2)]) / 2,
    (tt[n] - tt[n - 1]) / 2)
}

# criterion: F_linear(a, b) = int [x(a+bt) - x0(t)]^2 dt
Flinear <- function(t0, x0, t, x, a, b, ngrid = 200){
  tfine <- seq(t0[1], t0[length(t0)], length.out = ngrid)
  x0fine <- approx(t0, x0, xout = tfine, rule = 2)$y
  z <- a + b * tfine
  xfine <- approx(t, x, xout = z, rule = 1)$y
  xfine[is.na(xfine)] <- 0
  w <- trapw(tfine)
  sum((xfine - x0fine)^2 * w)
}

# gradient
FLgrad <- function(t0, x0, t, x, a, b, ngrid = 200){
  tfine <- seq(t0[1], t0[length(t0)], length.out = ngrid)
  x0fine <- approx(t0, x0, xout = tfine, rule = 2)$y
  z <- a + b * tfine
  xfine <- approx(t, x, xout = z, rule = 1)$y
  xfine[is.na(xfine)] <- 0
  tmid <- (t[-1] + t[-length(t)]) / 2
  Dx <- diff(x) / diff(t)
  dxfine <- approx(tmid, Dx, xout = z, rule = 1)$y
  dxfine[is.na(dxfine)] <- 0
  r <- xfine - x0fine
  w <- trapw(tfine)
  c(2 * sum(r * dxfine * w),
    2 * sum(r * dxfine * tfine * w))
}

# approximate Hessian (D^2x)
FLHess <- function(t0, x0, t, x, a, b, ngrid = 200){
  tfine <- seq(t0[1], t0[length(t0)], length.out = ngrid)
  z <- a + b * tfine
  tmid <- (t[-1] + t[-length(t)]) / 2
  Dx <- diff(x) / diff(t)
  dxfine <- approx(tmid, Dx, xout = z, rule = 1)$y
  dxfine[is.na(dxfine)] <- 0
  w <- trapw(tfine)
  h11 <- 2 * sum(dxfine^2 * w)
  h12 <- 2 * sum(dxfine^2 * tfine * w)
  h22 <- 2 * sum(dxfine^2 * tfine^2 * w)
  rbind(c(h11, h12), c(h12, h22))
}
```

```

}

# Newton-Raphson: interface matches problem specification
linearreg <- function(t0, x0, t, x, iter.max = 10){
  alpha <- 1
  tol <- 1e-5
  ngrid <- 200

  a <- 0
  b <- 1
  Fval <- Flinear(t0, x0, t, x, a, b, ngrid = ngrid)

  for(iter in seq_len(iter.max)){
    g <- FLgrad(t0, x0, t, x, a, b, ngrid = ngrid)
    H <- FLHess(t0, x0, t, x, a, b, ngrid = ngrid)

    H <- H + diag(1e-8, 2)      # numerical stabilization
    step <- drop(solve(H, g))

    stepsize <- alpha
    repeat{
      anew <- a - stepsize * step[1]
      bnew <- b - stepsize * step[2]

      if(bnew <= 0){            # keep b positive
        stepsize <- stepsize / 2
        if(stepsize < 1e-6) break
        next
      }

      Fnew <- Flinear(t0, x0, t, x, anew, bnew, ngrid = ngrid)

      if(Fnew <= Fval || stepsize < 1e-6) break
      stepsize <- stepsize / 2    # step-halving
    }

    if(stepsize < 1e-6) break

    delta <- abs(Fnew - Fval)     # convergence check BEFORE update
    a <- anew
    b <- bnew
    Fval <- Fnew

    if(delta < tol) break
  }

  c(a = a, b = b, objective = Fval)
}

```

1) 시뮬레이션 1: 잘 되는 경우 (affine time warp)

실제 affine warp ($a_{\text{true}}, b_{\text{true}}$) = (80, 0.85)로 생성한 데이터에 적용한다.

```
basefun <- function(s){
  exp(-((s - 250) / 80)^2) +
    0.7 * exp(-((s - 700) / 120)^2) +
    0.2 * sin(2 * pi * s / 300)
}

t <- seq(0, 1000, by = 1)
t0 <- seq(0, 1000, by = 2)

a.true <- 80
b.true <- 0.85

x0 <- basefun(t0)
x <- basefun((t - a.true) / b.true)

fit.good <- linearreg(t0, x0, t, x, iter.max = 20)
cat(sprintf("Estimated a = %.5f (true: %g)\n", fit.good["a"], a.true))

## Estimated a = 80.00026 (true: 80)

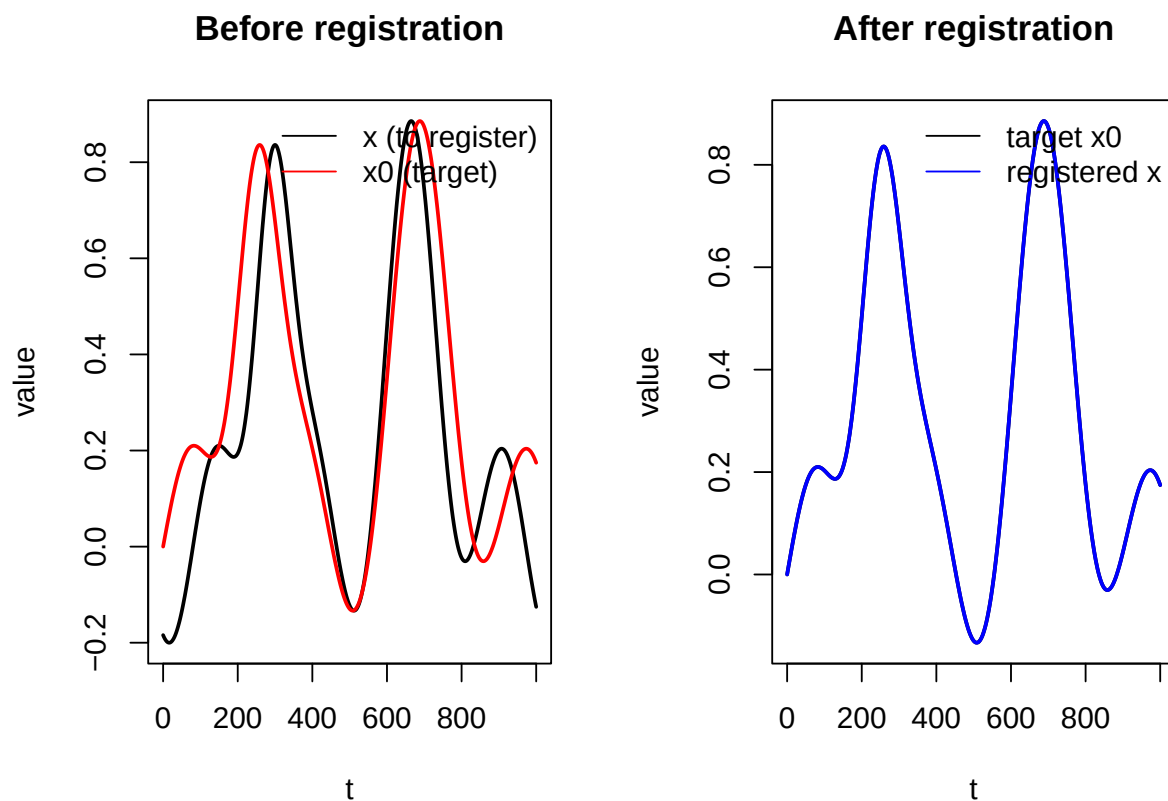
cat(sprintf("Estimated b = %.7f (true: %g)\n", fit.good["b"], b.true))

## Estimated b = 0.8499995 (true: 0.85)

tfine <- seq(min(t0), max(t0), length.out = 400)
x0fine <- approx(t0, x0, xout = tfine, rule = 2)$y
xreg <- approx(t, x, xout = fit.good["a"] + fit.good["b"] * tfine, rule = 1)$y
xreg[is.na(xreg)] <- 0

par(mfrow = c(1, 2))
plot(t, x, type = "l", lwd = 2,
     xlab = "t", ylab = "value", main = "Before registration")
lines(t0, x0, col = "red", lwd = 2)
legend("topright", legend = c("x (to register)", "x0 (target)"),
     col = c("black", "red"), lty = 1, bty = "n")

plot(tfine, x0fine, type = "l", lwd = 2,
     xlab = "t", ylab = "value", main = "After registration")
lines(tfine, xreg, col = "blue", lwd = 2)
legend("topright", legend = c("target x0", "registered x"),
     col = c("black", "blue"), lty = 1, bty = "n")
```



$\hat{a} \approx 80.0$, $\hat{b} \approx 0.85$ 로 실제값 $(80, 0.85)$ 를 거의 정확히 복원했다. 곡선에 뚜렷한 peak와 valley가 여러 개 존재하고 실제로 affine time warp로 생성된 경우 linear registration이 매우 잘 작동함을 확인할 수 있다.

2) 시뮬레이션 2: 잘 안 되는 경우 (amplitude 차이만 있는 경우)

이번에는 time warp는 없고 amplitude만 다른 데이터를 생성한다.

```
x0.bad <- basefun(t0)
x.bad  <- 1.4 * basefun(t)

fit.bad <- linearreg(t0, x0.bad, t, x.bad, iter.max = 20)
cat(sprintf("Estimated a = %.6f (reference: identity 0)\n", fit.bad["a"]))

## Estimated a = -11.617429 (reference: identity 0)

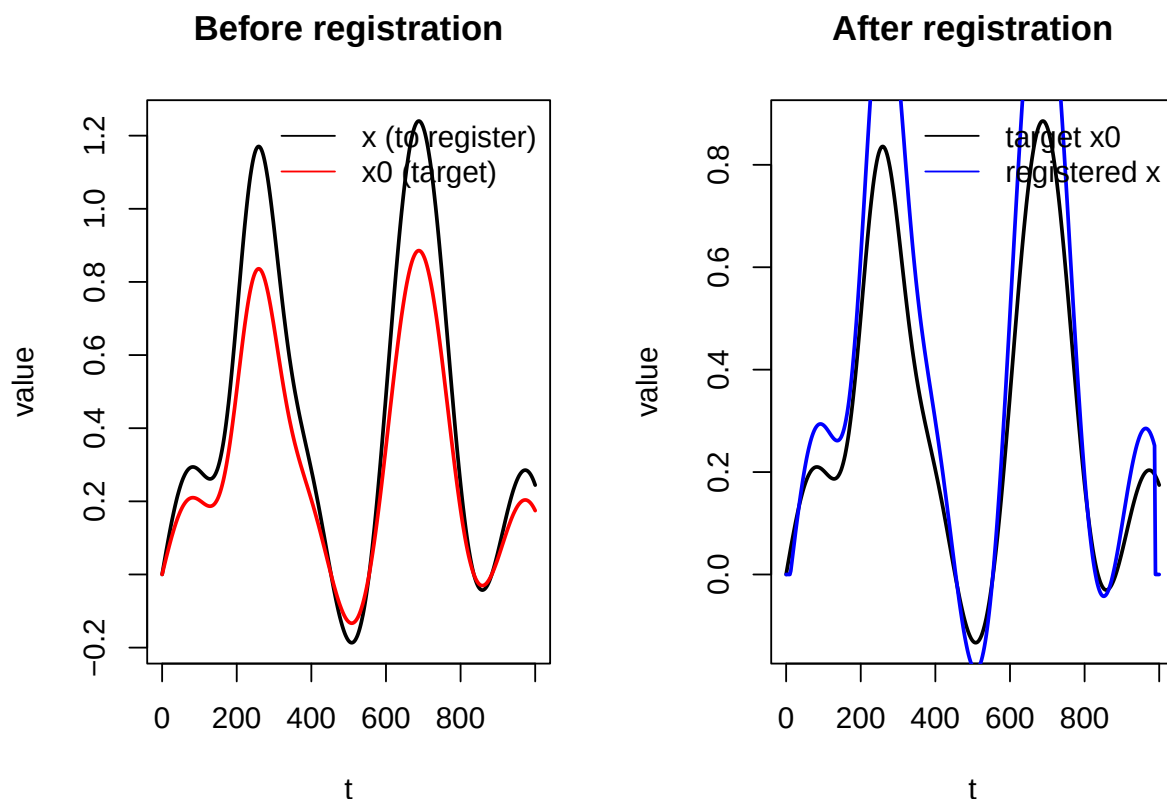
cat(sprintf("Estimated b = %.6f (reference: identity 1)\n", fit.bad["b"]))

## Estimated b = 1.021888 (reference: identity 1)

x0fine.bad <- approx(t0, x0.bad, xout = tfine, rule = 2)$y
xreg.bad   <- approx(t, x.bad, xout = fit.bad["a"] + fit.bad["b"] * tfine, rule = 1)$y
xreg.bad[is.na(xreg.bad)] <- 0

par(mfrow = c(1, 2))
plot(t, x.bad, type = "l", lwd = 2,
     xlab = "t", ylab = "value", main = "Before registration")
lines(t0, x0.bad, col = "red", lwd = 2)
legend("topright", legend = c("x (to register)", "x0 (target)"),
     col = c("black", "red"), lty = 1, bty = "n")

plot(tfine, x0fine.bad, type = "l", lwd = 2,
     xlab = "t", ylab = "value", main = "After registration")
lines(tfine, xreg.bad, col = "blue", lwd = 2)
legend("topright", legend = c("target x0", "registered x"),
     col = c("black", "blue"), lty = 1, bty = "n")
```



Phase difference가 없으므로 time warping 자체는 불필요한 상황이며, no-warp reference는 identity transformation $(a, b) = (0, 1)$ 이다. 그러나 F_{linear} 는 amplitude 차이와 phase 차이를 완전히 분리하지 못하므로, amplitude-only 차이가 있는 경우에도 criterion이 시간축을 왜곡하는 방향으로 (a, b) 를 조정할 수 있다. 실제로 추정된 (a, b) 는 identity reference $(0, 1)$ 에서 벗어났으며, 이는 least-squares 기반 registration의 한계를 보여준다.

구현 이슈 및 한계

Newton-Raphson step을 그대로 적용하면 $b \leq 0$ 이 나올 수 있다. Affine warping이 시간 순서를 보존하려면 $b > 0$ 이어야 하므로, 구현에서는 step-halving으로 $b > 0$ 을 유지했다. 또한 곡선의 기울기가 약하거나 feature가 부족하면 Hessian이 거의 singular해질 수 있으므로, 작은 ridge term을 더해 수치적으로 안정화했다.

이 방법은 affine time warp가 실제로 타당하고 곡선에 식별 가능한 peak/valley가 있을 때 가장 잘 동작한다. 반대로 amplitude 차이가 크거나 feature가 모호한 경우에는 추정된 a, b 가 왜곡될 수 있다.

Disclosure of LLM Use

본 과제 작성 과정에서 코드 디버깅 보조를 위해 LLM을 참고하였다. 최종 수식 전개, 코드 작성, 코드 실행 결과 확인, 해석 문장 수정은 수업 자료와 R 실행 결과를 바탕으로 직접 검토하였다.